

Build the AdHush Box

A little box that sits between your cable box and your TV, **watches for commercials, and clicks the sound off** until your show comes back. No soldering. About an afternoon. Around **\$120–160** in parts. If you can build a LEGO set from instructions and type commands carefully, you can build this.

5 VOLT RELAY EDITION — the kind stores actually stock

⚡ Safety rules — read these first

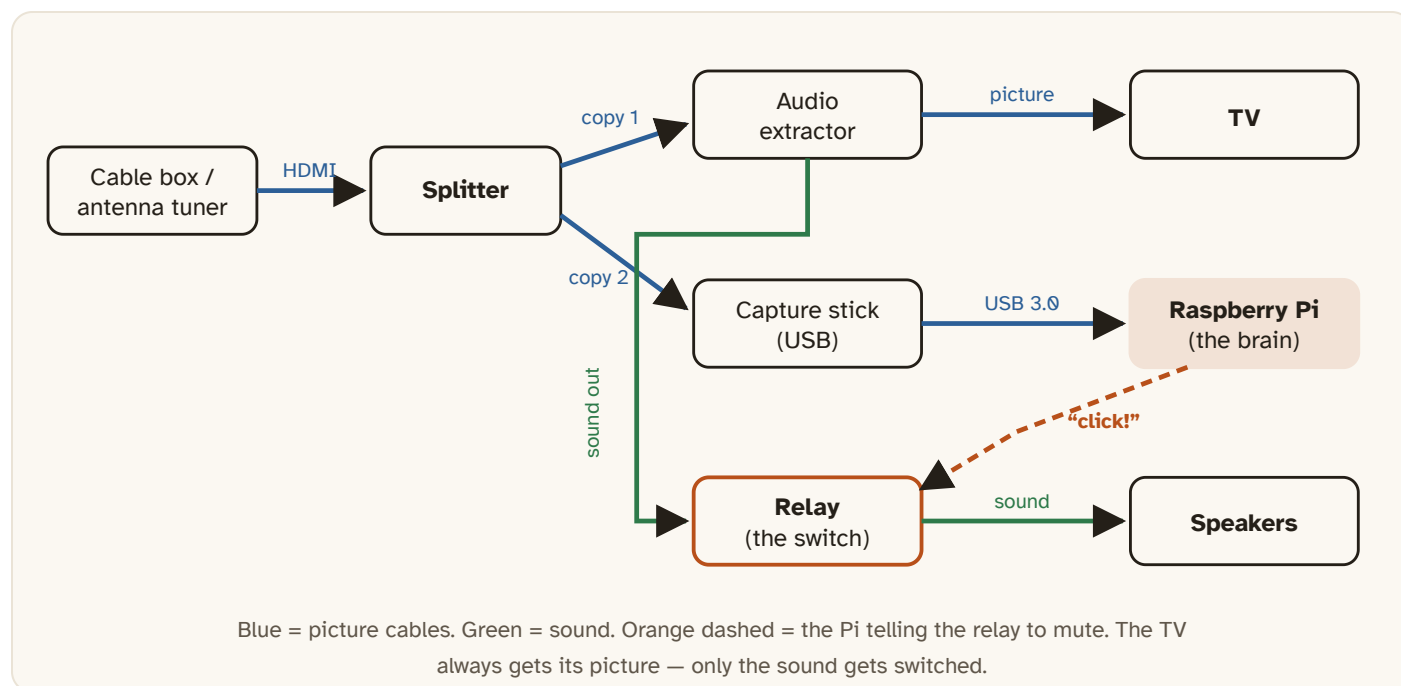
Everything in this project runs on **low-voltage USB power**. The biggest voltage you will ever touch is 5 volts — about the same as a phone charger cable.

Never open, cut, or wire anything that plugs straight into a wall outlet. The only cable you cut in this whole build is a thin headphone-style audio cable, and it must be *unplugged from everything* while you cut it.

Where you see the **ADULT CHECK** tag, stop and have a grown-up look at your work *before* you plug in any power.

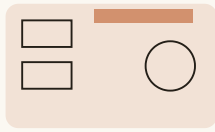
★ The big picture

The TV signal gets **split into two copies**. One copy goes to your TV like normal. The other copy goes to the Raspberry Pi, which watches it. The **sound** takes a detour through a little switch called a **relay** — and the Pi clicks that switch whenever a commercial starts.



— picture (HDMI / USB) — sound — control signal — 5 V power

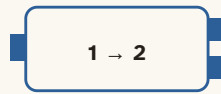
The parts



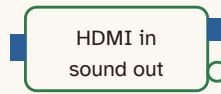
1 · Raspberry Pi 4



2 · microSD 32 GB



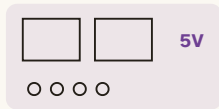
3 · HDMI splitter



4 · Audio extractor



5 · Capture stick



6 · Relay module
2-channel, 5 volt



7 · Jumper wires
kit + mini breadboard



8 · 3.5 mm audio
cables ×2 (one gets cut)



9 · Short HDMI
cables ×2



10 · Small screwdriver,
scissors, tape

Lay everything out before you start, like the first page of a LEGO manual. Checking the list now is much easier than finding a missing part halfway through.

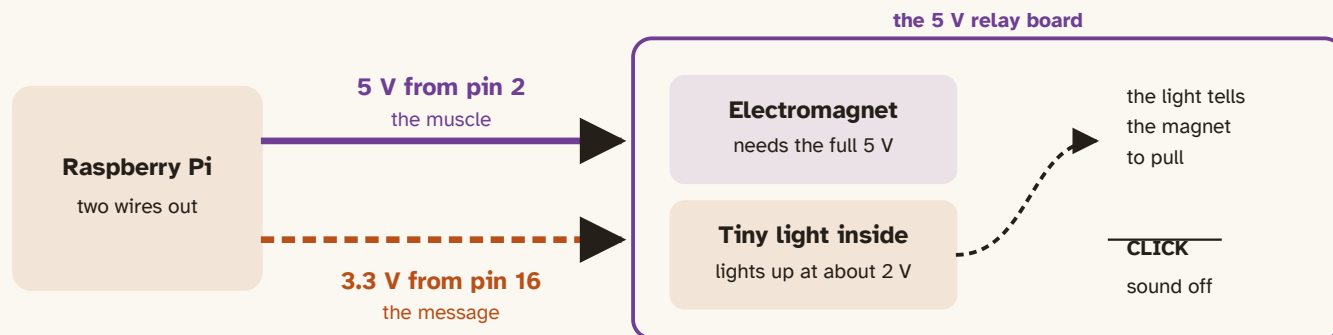
Part	What it does	≈ Price
Raspberry Pi 4 (2 GB or more) + official USB-C power supply	The brain — a tiny computer	\$45–60
microSD card, 32 GB	The brain's memory	\$8
Powered HDMI splitter, 1 in → 2 out	Makes a copy of the picture	\$15
HDMI audio extractor (HDMI in → HDMI out + headphone/RCA out)	Pulls the sound onto a normal audio cable	\$15–20
USB HDMI capture stick (must say “UVC”; USB 3.0 is best)	Lets the Pi see the copied picture	\$8–25
Relay module, 2-channel, 5 V, with screw terminals — opto-isolated is best. This is the kind hobby shops keep in stock.	The click-switch that cuts the sound	\$7
Jumper wire kit with a mini breadboard — you will use 2 female-to-female and 3 male-to-female wires	Connect the Pi to the relay, and send the one mute signal to <i>both</i> relay channels (Step 5 explains)	\$8–10
Two 3.5 mm audio cables (one gets cut in half) — or, to skip the cutting, two “3.5 mm plug to screw terminal” adapters	Carry the sound	\$6
2 short HDMI cables	Connect the boxes	\$10
Small screwdriver, scissors, electrical tape	Tools you probably already have	—

Borrow for setup day: a keyboard, a mouse, a monitor or spare TV, and your Wi-Fi password. Speakers or a soundbar with an aux input sound best; your TV's own “audio in” jack works too.

5V Why a 5 volt relay?

Older versions of this guide asked for a **3.3 volt** relay module, because 3.3 volts is what the Pi's little signal pins put out. The trouble is that almost nobody sells 3.3 V modules in a store — walk into a hobby shop and every relay board on the shelf says **5 V**.

Good news: a 5 V board works, and this guide is written for it. Here is the trick — a relay board has **two separate jobs**, and they need different amounts of electricity.



The **5 V wire** does the heavy lifting. The Pi's 3.3 V signal wire only has to light a tiny lamp inside the board, and that lamp needs about 2 volts — so 3.3 volts is plenty. That is why a 5 V board works from a 3.3 V Pi.

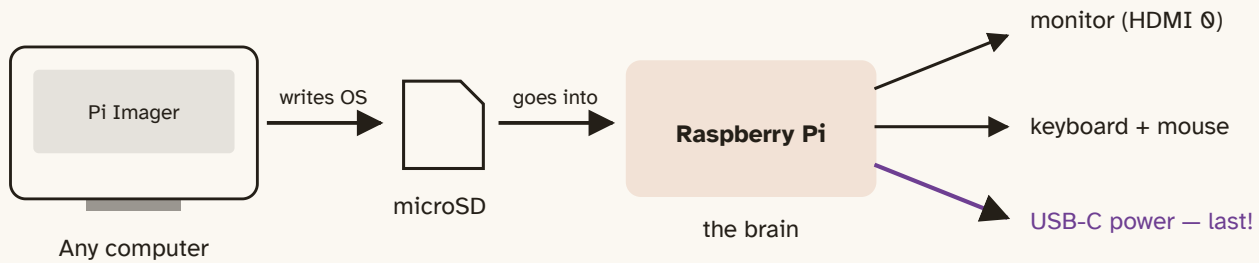
What changed from the old 3.3V guide

	Old 3.3V guide	This 5V guide
Relay module	3.3 V logic (hard to buy)	5 V (on every shelf)
VCC wire	pin 1 — 3.3 V	pin 2 or pin 4 — 5 V
GND, signal, sound wiring	pin 6, pin 16, COM + NC	all unchanged
Settings	<code>active_high = true</code>	the same, unless the test comes out backwards

1

Wake up the Pi

ADULT CHECK



Setup day only: the monitor, keyboard, and mouse all go back when you're done. Plug the power in *last* — a Pi turns on the moment it gets power.

1. On a computer, install **Raspberry Pi Imager** (free, from raspberrypi.com). Put in the microSD card and write **Raspberry Pi OS** to it. In the imager's settings gear, set a username, a password, and your Wi-Fi name and password.
2. Move the card into the Pi. Connect the monitor, keyboard, and mouse. Power last. Wait for the desktop to appear.
3. Open the black **Terminal** window and type these four lines, pressing Enter after each one. Type them exactly — capital letters matter:

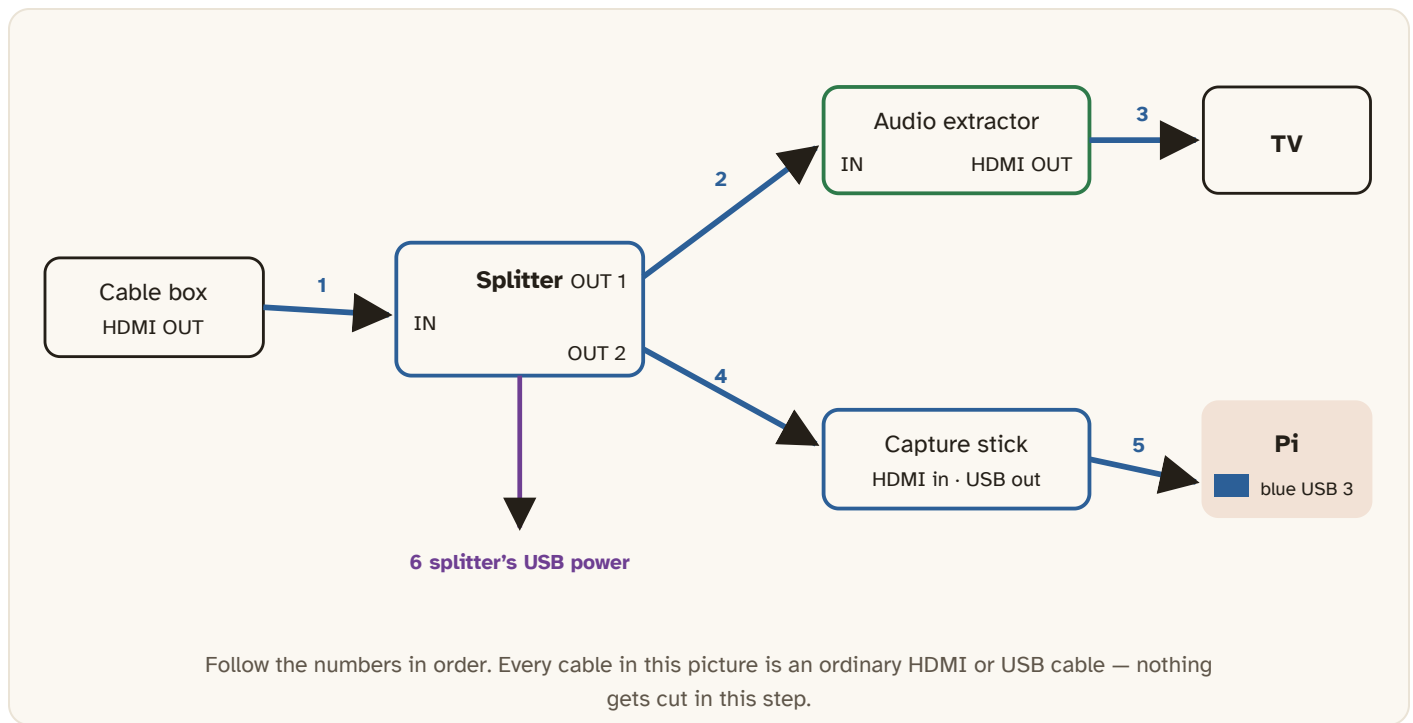
```
git clone https://github.com/socrtwo/adhush
cd adhush
sudo scripts/install-pi.sh
cp config/adhush-passthrough.example.toml config/adhush.toml
```

That last line makes the box's settings file. Leave it alone for now — the defaults match this build. You'll come back to it in Step 6 only if the relay turns out to click backwards.

2

Build the picture path

No tools needed — this step is all plugging in cables. You are making the copy of the TV signal that the Pi will watch.



1. Unplug the HDMI cable that runs from your **cable box to your TV**, and plug that end into the **splitter's IN** instead.
2. New cable: splitter **OUT 1** → audio extractor **IN**.
3. New cable: extractor **HDMI OUT** → your **TV**. (Picture is back!)
4. New cable: splitter **OUT 2** → the **capture stick**.
5. Plug the capture stick into a **blue USB 3.0** port on the Pi.
6. Connect the splitter's little USB power cord.

Test it: turn on the TV. You should see your channels exactly like before. Black screen? Try the splitter's other output, or swap in a different HDMI cable.

Then tell the Pi which gadget is the sound

The Pi can hear from more than one place — its own headphone jack, the capture stick, anything else that is USB. It needs to know which one is the stick. In the terminal, inside the `adhus` folder, type:

```
adhus doctor
```

Near the bottom it prints a list of **sound cards**. It looks like this:

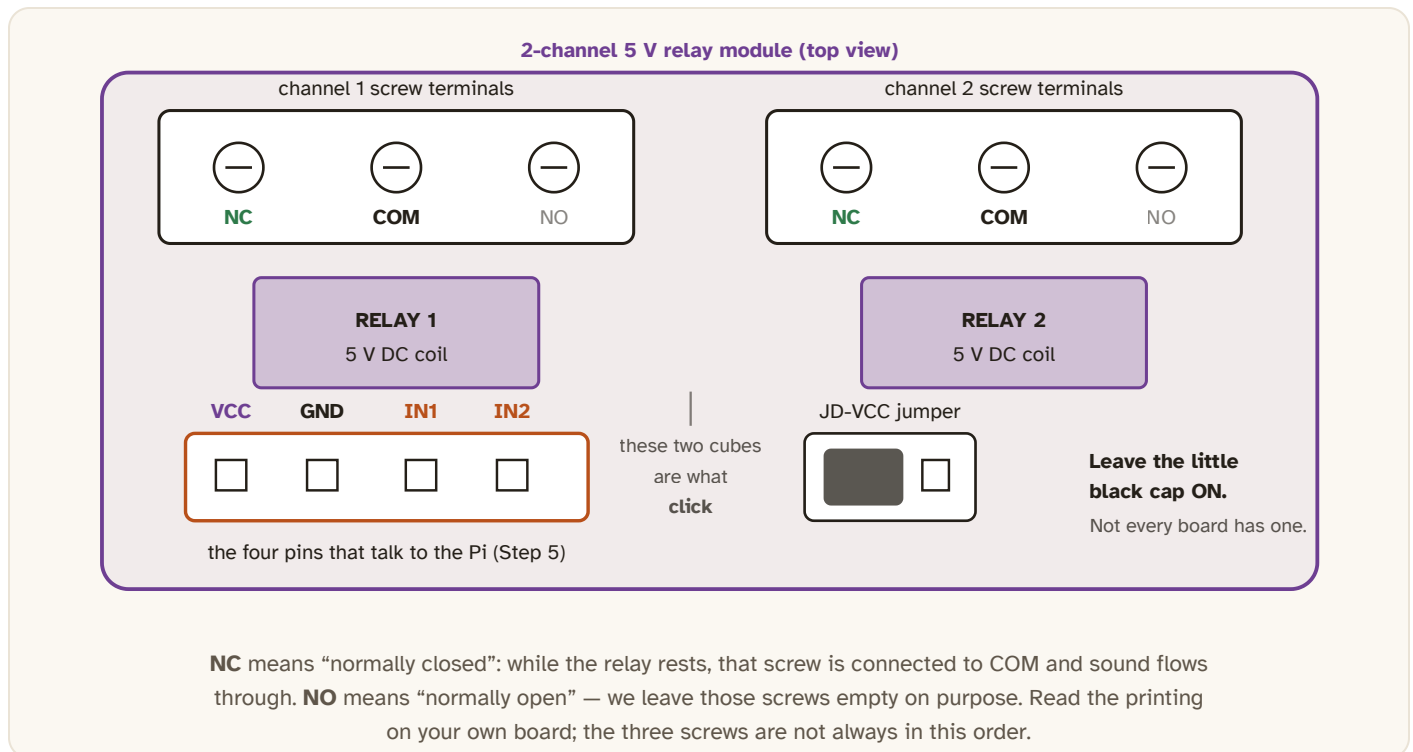
```
sound cards – put the capture stick's card in capture.audio_device:
card 0: Headphones      bcm2835_Headpho - bcm2835 Headphones
      use alsa:hw:0,0    (or alsa:hw:CARD=Headphones,DEV=0, which survives reboots)
card 1: MS2109          USB-Audio - MS2109 <- USB, probably the stick
      use alsa:hw:1,0    (or alsa:hw:CARD=MS2109,DEV=0, which survives reboots)
video devices – capture.device: /dev/video0 /dev/video1
```

1. Find the card marked **USB, probably the stick** and read its number. On most Pis it is **card 1** — and that is what the settings file already says, so you are done.
2. If your number is different: open the settings with `nano config/adhush.toml`, find the line that starts `audio_device =`, and change the `1` in `"alsa:hw:1,0"` to your number. Save with **Ctrl+O**, Enter, then **Ctrl+X**.
3. Run `adhush doctor` again. The line `audio device alsa:hw:... present` should now say `ok`.

If the box ever goes deaf after a reboot, USB gadgets sometimes swap numbers. Paste the name form instead — the one like `alsa:hw:CARD=MS2109,DEV=0` — and it stays put.

3 Meet your relay board

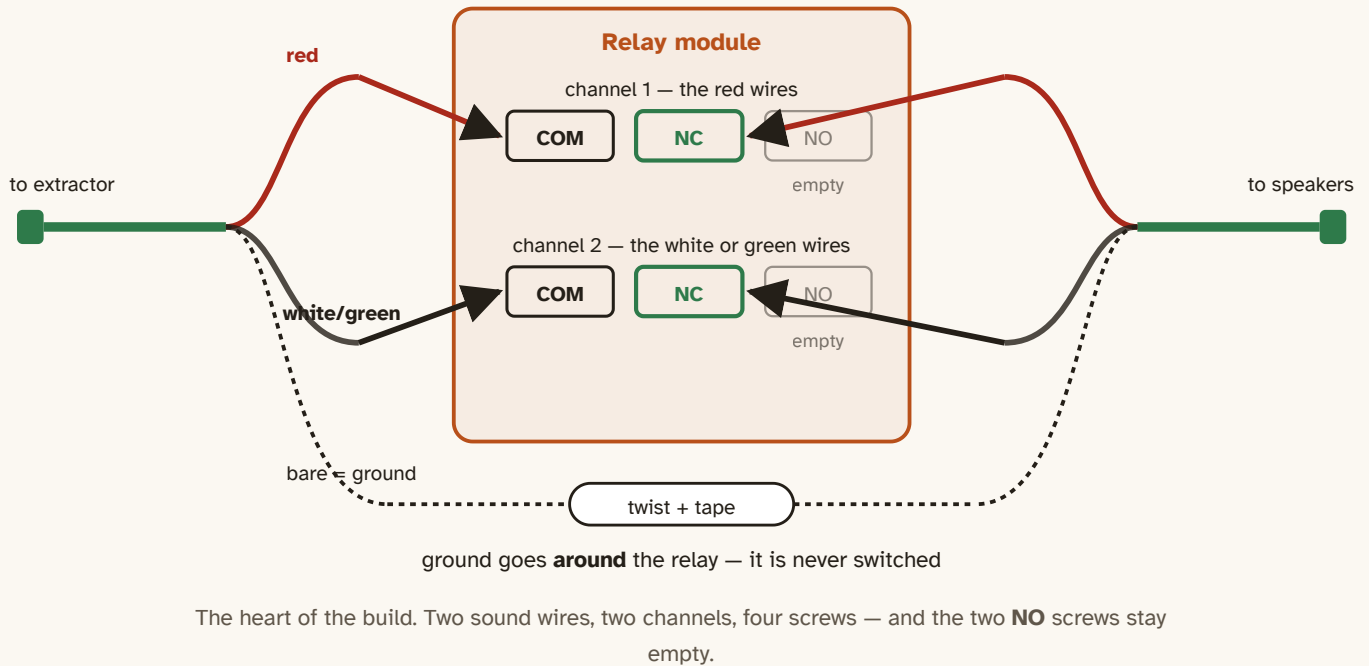
Before any cutting, get to know the board. A 2-channel 5 V relay module has **two sides**: a *low-voltage side* where four little pins talk to the Pi, and a *switch side* with fat screw terminals where your sound wires go. The two sides never touch electrically — that is exactly why this is safe.



The one rule that makes this safe to leave running

Sound goes through **COM + NC**, never NO. That way, if the Pi crashes, loses power, or you unplug the whole thing, the relay falls back to resting and your sound comes **back on**. A broken AdHush box should be a box that does nothing — not a box that leaves your TV silent.

Now the scissors. Sound will travel: extractor → **through the relay** → speakers. Make sure the cable you are about to cut is unplugged from everything first.



1. Cut one audio cable in the middle. Strip about 2 cm of the outer cover off each cut end. Inside are three small wires: one **red**, one **white or green**, and a bare copper one. The bare one is **ground**. (Some cables use a copper mesh instead; gather it and twist it into a strand.)
2. Look closely at the strands. If they are shiny and look painted, they are — scrape the last 1 cm with sandpaper or the back of a knife until dull copper shows. Painted strands will not make contact in a screw terminal.
3. Twist the two **ground** wires together and wrap the joint in tape. Ground is never switched.
4. Screw the **red** wires into channel 1's **COM** and **NC**. It does not matter which red goes to which of those two.
5. Screw the **white/green** wires into channel 2's **COM** and **NC**. Leave every **NO** screw empty.
6. Give each wire a gentle tug. If one pulls out, the screw isn't tight enough.
7. Plug one half of the cut cable into the **extractor's audio out** and the other half into your **speakers**.

No-scissors option

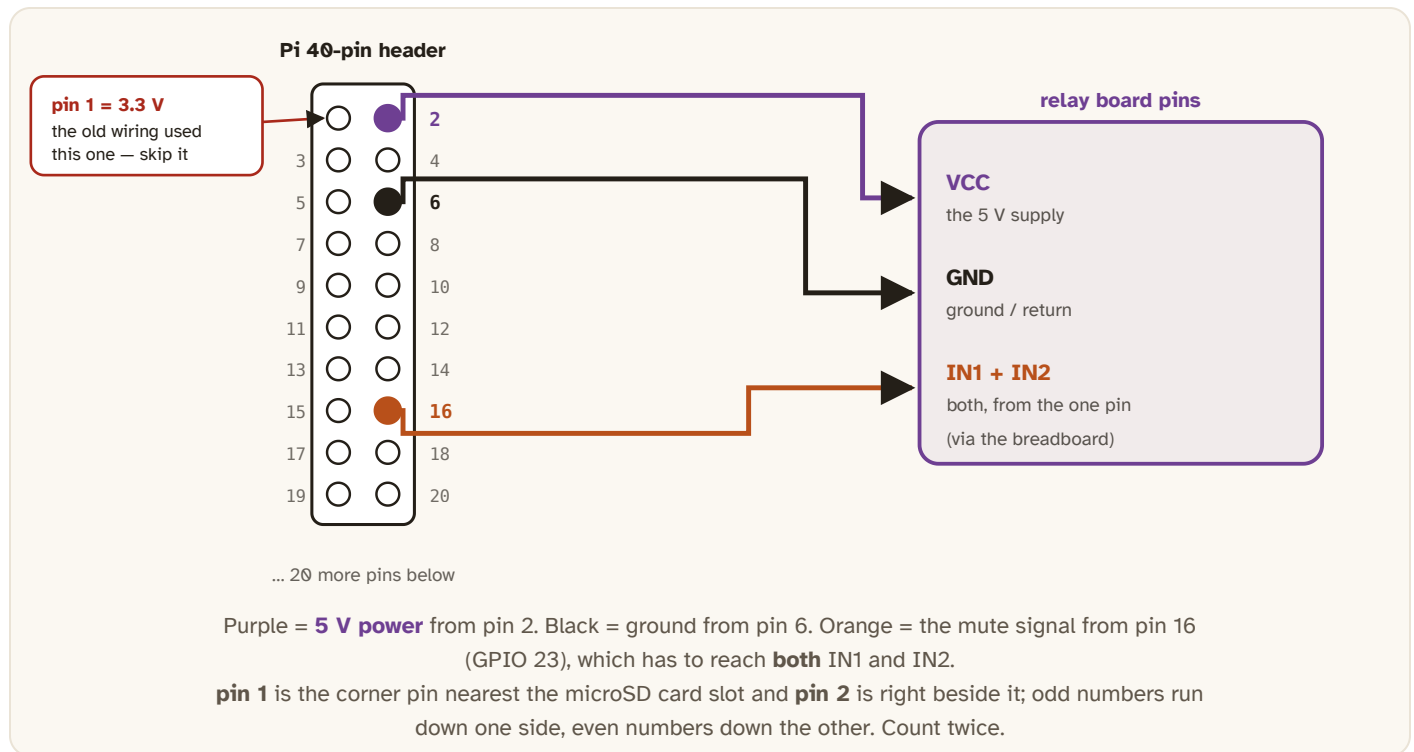
Two "**3.5 mm plug to screw terminal**" adapters (about \$6 a pair) do the same job as the cut cable, with nothing to cut or scrape. Plug one adapter into the extractor and one into the speakers. Each has three screws: **L**, **R**, and **G** (ground). Run short wires L→relay channel 1→L, R→channel 2→R, and join the two G screws to each other with one wire. That is the whole sound path.

5

Connect the Pi to the relay

ADULT CHECK

Five little jumper wires and a breadboard, and this is the part where the **5 volt** change matters. Power comes from the Pi's **5 V pin**. The go/stop signal comes from **GPIO 23**, which lives on **physical pin 16**.



Pi pin	What it is	Goes to
pin 2	5 V power (pin 4 is also 5 V — either works)	VCC
pin 6	Ground	GND
pin 16	GPIO 23 — the mute signal, 3.3 V	IN1 and IN2
pin 1	3.3 V — what the old 3.3 V relay used	nothing

Getting one signal into two IN pins

Both channels need the *same* signal, because left and right sound must switch at the same instant. But one Pi pin only holds one jumper socket. So the signal takes a hop through the **mini breadboard**: one male-to-female wire from pin 16 into any breadboard row, then two more male-to-female wires from that *same row* out to IN1 and IN2. Every hole in a row is joined inside the board, so all three wires are connected. (A **1-channel DPDT** board, if you find one, switches both sound wires from a single IN pin and needs no breadboard.)

ADULT CHECK before power-on

Read this list out loud with a grown-up, pointing at each thing:

- The sound wires are on **COM** and **NC**. Both **NO** screws are empty.
- The two ground wires are twisted together and taped.
- VCC goes to **pin 2** (or pin 4) — **5 V**. Not pin 1.
- GND goes to **pin 6**. The signal goes to **pin 16**.
- No bare metal is touching any other bare metal anywhere.
- The little black **JD-VCC cap**, if your board has one, is still on.

Then power up the Pi. You should hear the TV through your speakers. Turn the TV's own volume all the way down — the relay path is the sound now.

6 Wake it up

In the Pi's terminal, inside the `adhush` folder, run these one at a time. Start with the relay test — it is the fastest way to know your wiring is right.

```
adhush probe --active  # sound should cut out for 2 seconds, then come back
adhush calibrate       # run this while a SHOW is on, not an ad
adhush run             # start watching!
```

During `adhush probe --active` you should hear one *click*, two seconds of silence, then another *click* and the sound returns. That is the whole build, proven.

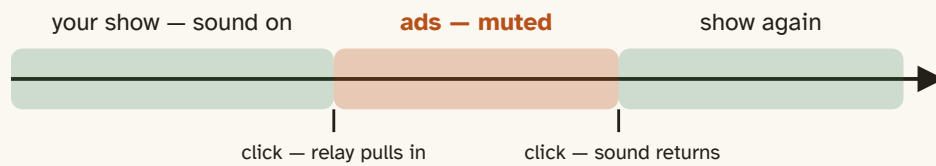
If the relay works backwards — the one 5V gotcha

Some 5 V boards mute when the signal goes *high*. Others mute when it goes *low*. You cannot tell from the outside, so you test. If your sound is **off at rest** and comes **on** during the two-second test, your board is the other kind. One-line fix, not a wiring mistake.

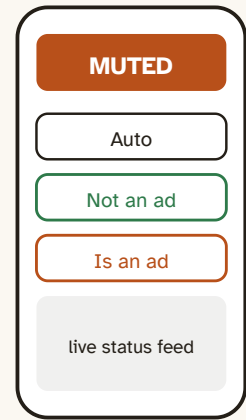
Open the settings file: `nano config/adhush.toml`, find the `[control.relay_hdmi]` part, and change `true` to `false`:

```
[control.relay_hdmi]
gpio = 23
active_high = false
```

Save with **Ctrl+O**, then Enter, then **Ctrl+X** to leave. Run `adhush probe --active` again — now it should be the right way round.



slow to mute, quick to un-mute — on purpose.
Missing the first second of an ad is fine.
Missing the first second of your show is not.



your phone

What success sounds like — and your phone remote. Open `platforms/web/index.html` on any device on your Wi-Fi to watch PROGRAM / MUTED live and press the two teaching buttons.

- **“Not an ad”** — press it if the box mutes your actual show. Sound comes back instantly *and* it forgets the mistake.
- **“Is an ad”** — press it during a commercial it missed, and it memorizes that ad for next time.
- The first evening it is mostly learning, and it gets noticeably sharper on ads it has heard before. To make the box start by itself whenever it has power, run `sudo systemctl enable --now adhush`.

If something's wrong

Problem	Try this
No picture on the TV	Reseat the HDMI cables; check the splitter's power; swap OUT 1 and OUT 2.
No sound at all	Are both wires on COM + NC (not NO)? Are the grounds twisted together? Is each cable half plugged into the extractor and the speakers?
Never hears anything, or <code>doctor</code> says the audio device FAILS	Wrong card number. Re-read the sound-card list in <code>adhush doctor</code> and fix <code>audio_device</code> (end of Step 2).
Sound only on one side	Only one channel is wired, or one screw is loose. Channel 1 carries red, channel 2 carries white/green.
Muted at rest, sound during the test	Your board is the other kind. Set <code>active_high = false</code> — see the <i>relay works backwards</i> box in Step 6.
Relay never clicks at all	Run <code>sudo systemctl start pigpiod</code> , then <code>adhush probe --active</code> again. Check VCC is on pin 2 , not pin 1 — a 5 V board will not pull in on 3.3 V.
Relay buzzes or chatters	Not enough current. Use the official Pi power supply and keep the JD-VCC cap on. If it still buzzes, that board wants a 3.3 V-logic module instead — not your fault.
It mutes my show!	Press Not an ad on the phone page — it learns from that.
<code>adhush run</code> shows an error	Read the message, then run <code>adhush doctor</code> — it lists exactly what is missing.

One honest note. A few channels and streaming boxes copy-protect their HDMI signal, so the capture stick sees nothing. AdHush never tries to break that protection. For those sources the box still works in listen-only mode, using just the sound to spot ads.

AdHush — open source under the MIT license · github.com/socrtwo/adhush

Plain-text version: [docs/build-guide-beginner.md](#) · wiring detail and the reasoning behind the fail-with-sound design: [docs/hardware-passthrough-box.md](#) · building with an Arduino, ESP32, or Pico instead: [docs/build-guide-microcontrollers.md](#).

AdHush mutes your own speakers in your own home. It does not modify, copy, record, or redistribute anything you watch, and it never tries to defeat content protection.